CS315 · ALGORITHMS ANALYSIS & DESIGN

Midterm 472 Solutions

Midterm 472 · from L1 to L7

Muharib

01 Solution Q1: matrix chain

The idea: multiplying a `p × q` matrix by a `q × r` matrix costs `p·q·r` operations. We want the parenthesization with the smallest total. We fill the table from short chains to long chains.

```
m[i, j] = min over k of:   m[i, k] + m[k+1, j] + cost
```

```
cost = (rows of i) × (cols of k) × (cols of j)
```

1 Chains of two matrices

```
AB  =  4 × 5 × 8  =  160
```

```
BC  =  5 × 8 × 3  =  120
```

```
CD  =  8 × 3 × 7  =  168
```

2 Chains of three matrices

```
ABC:   A(BC)  =  0 + 120 + 4 × 5 × 3  =  180     ← min
       (AB)C  =  160 + 0 + 4 × 8 × 3  =  256
BCD:   B(CD)  =  0 + 168 + 5 × 8 × 7  =  448
       (BC)D  =  120 + 0 + 5 × 3 × 7  =  225     ← min
```

3 The full chain ABCD

```
A(BCD)    =  0 + 225 + 4 × 5 × 7    =  365
(AB)(CD)  =  160 + 168 + 4 × 8 × 7  =  552
(ABC)D    =  180 + 0 + 4 × 3 × 7    =  264     ← min
```

	A	B	C	D
A	0	160	180	**264**
B	—	0	120	225
C	—	—	0	168
D	—	—	—	0

```
Minimum operations = 264
```

```
Best order of multiplication:   ((A(BC))D)
```

The order comes from tracing the choices back: ABCD split as `(ABC)D`, and ABC split as `A(BC)`.

02 Solution Q2: True / False + MCQ

#	Answer	Reason
1	**True**	A character that occurs more often gets a shorter code.
2	**False**	Greedy takes the best current choice, which does not give the optimum for every DP problem. Matrix chain and LCS, for example, cannot be solved greedily.
3	**False** *	The slide writes only `T(MERGE)`, and the merge works on `n` elements, not $\frac{n}{2}$.
4	**False**	The same environment **and the same inputs** must be used.
5	**True**	The words of the Lecture 1 slide.
6	**c** *	Dynamic programming stores the results of sub-problems and reuses them.

About the two stars

Item 3: the statement changes the last term of the slide's equation. It is most likely false, but without an answer key I cannot be certain. Ask the instructor.

Item 6: the wording of this copy contradicts itself ("does NOT use the past results and uses them"). The original 451 midterm reads "An algorithm which uses the past results and uses them to find new results", and its answer is c. The word NOT is evidently a retyping mistake.

03 Solution Q3: LCS

We build a table whose rows are the letters of `X` and whose columns are the letters of `Y`, with a first row and column of zeros. Each cell is computed like this:

```
X[i] = Y[j]:   c[i, j] = c[i−1, j−1] + 1        diagonal + 1
```

```
X[i] ≠ Y[j]:   c[i, j] = max(top, left)
```

	∅	A	B	C	A	D	F
∅	0	0	0	0	0	0	0
A	0	**1**	1	1	1	1	1
C	0	1	1	2	2	2	2
B	0	1	**2**	2	2	2	2
A	0	1	2	2	**3**	3	3
E	0	1	2	2	3	3	3
D	0	1	2	2	3	**4**	4

```
Length = 4
```

```
The subsequence:   1)  ABAD      2)  ACAD
```

The length is the last cell of the table. We recover the letters by walking back from the last cell: when the two letters match we take the letter and move diagonally, otherwise we move to the larger of top and left. The coloured cells are the path of `ABAD`. When top and left are equal we may go either way, which is where the second answer `ACAD` comes from.

```
Check:   X = A C B A E D   →   A . B A . D   and   A C . A . D
```

```
         Y = A B C A D F   →   A B . A D .   and   A . C A D .
```

04 Solution Q4: hash table

Key	4	13	49	6	40	33
k mod 9	4	4	4	6	4	6

Index	Chain
0	ø
1	ø
2	ø
3	ø
4	4 → 13 → 49 → 40
5	ø
6	6 → 33
7	ø
8	ø

Four keys leave remainder 4 when divided by 9, so they all sit in cell 4 in the order they arrived: `4 → 13 → 49 → 40`. Cell 6 holds `6 → 33`. The other cells are empty.

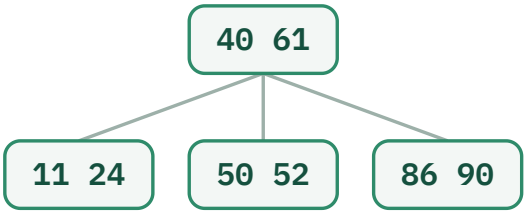
05 Solution Q5: B-tree

A B-tree of order 5: every node has at most 4 keys and at most 5 children. Every node other than the root has at least 2 keys. When a node reaches 5 keys it splits: the middle key moves

up to the parent, and the rest splits into two halves.

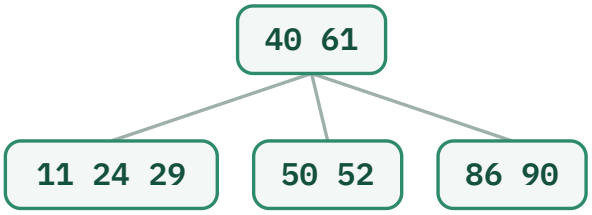
A Building the tree

```
Insert 24, 52, 90, 40      →   one node    [24, 40, 52, 90]
Insert 11                  →   5 keys, split, 40 moves up
Insert 86, 61              →   right leaf    [52, 61, 86, 90]
Insert 50                  →   5 keys, split, 61 moves up
```



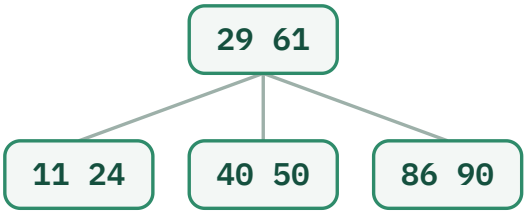
B Insert 29

`29` goes to the left leaf. It now has 3 keys, so there is no split.



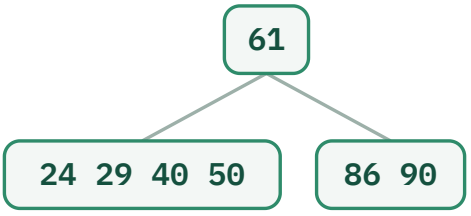
C1 Delete 52

The leaf of `52` is left with one key, below the minimum (underflow). The left sibling has 3 keys, so we borrow from it: its largest key moves up to the parent, and the parent's key moves down into the leaf.



C2 Delete 11

The leaf of `11` is left with one key. Its sibling has only 2 keys, so we cannot borrow. We merge: the two leaves and the parent key between them become one node.



The question says "order 5, min degree 2". I read it as at most 5 children and at least 2 keys per node. If the instructor's slides define the order differently, the tree will change.